CEP — TOPIC APPROVAL

COMPLEX ENGINEERING PROBLEM · CEP-2026

Indirect Denial-of-Service via Health-Check Manipulation in Docker Swarm

A Restart-Storm Availability Attack on Containerized Microservices

Investigating how Docker Swarm's restart_policy amplifies dependency failures into full service outages — and how to detect and mitigate them.



COURSE

Course Code & Title: Information Security

GROUP MEMBERS

· Muhammad Abdullah Khan — SE-23061
· Masroor Hassan — SE-23064
· Furqanullah — SE-23063
· Haris Masood — SE-23088
· Muhammad Huzaifa Hassan — SE-23089
· Muhammad Rafay — SE-23100

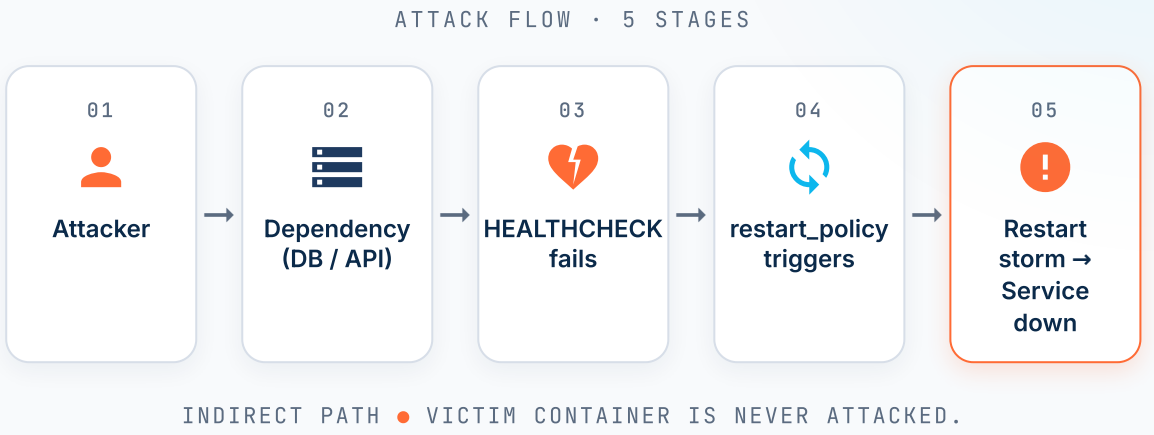
Instructor: Miss Sheerina

Problem Description & Motivation

In Docker Swarm, every service is monitored by a single HEALTHCHECK that drives BOTH traffic routing AND container restart decisions. When this check depends on an external resource — a database connection, downstream API, or cache — the orchestrator conflates dependency health with process health.

An attacker who merely slows that dependency (without ever touching the victim container) causes the health-check to fail repeatedly. Swarm's restart_policy, behaving exactly as designed, then enters a continuous terminate-and-restart cycle — a restart storm — that renders the service unavailable even though the application code itself is fully functional.

> **No exploit on the victim container is required — only its dependency needs to be slowed.**



Relevance to Containerized Environments

Why this problem is uniquely a containerization problem, not a generic web-app issue.

TILE 01

Orchestrator Coupling

Swarm's single health-check drives BOTH traffic routing AND restart decisions. There is no separation of concerns — one failed probe triggers two independent corrective actions simultaneously.

TILE 02

restart_policy Amplification

Auto-restart turns a transient dependency blip into a sustained outage. The orchestrator actively degrades availability while attempting to restore it — a self-inflicted DoS.

TILE 03

Microservice Blast Radius

One degraded dependency cascades to every service whose health-check touches it. A single slow database can take down an entire request path in seconds.

TILE 04

Default-Config Exposure

The vulnerable pattern — health-check that pings the database — is the documented best practice in tutorials and Helm charts. The risky configuration ships by default.

Summary — Container orchestrators turn dependency failures into availability failures. This class of risk is absent from monolithic deployments where health and routing are decoupled.

Proposed Approach — Detection & Mitigation

A monitoring-led detection layer plus a defense-in-depth mitigation strategy.

APPROACH
§ 4 · TWO-COL

DETECTION

LAYER 01

- Stream `Docker event API` + `inspect API` in real time.
- Detect health-state transitions: `starting → unhealthy → starting` cycles within a sliding window.
- Track restart frequency per service — alert when `restart_count` crosses threshold (e.g. `>5 restarts / 2 min`).
- Signature signal: divergence between application log success and health-check failure — the fingerprint of indirect DoS.

MITIGATION

LAYER 02

- Decouple health-checks from fragile external dependencies — split into liveness vs readiness probes with separate thresholds.
- Introduce circuit-breaker logic around dependency calls — transient failures no longer fail the health-check.
- Tune `restart_policy` with `max_attempts` + exponential backoff window to prevent restart storms.
- Where possible, run a local-only liveness probe (process + memory) that does NOT touch external resources.

LAB

Controlled Docker Swarm testbed with synthetic dependency latency injector.

METRICS

`MTTR · restart_count · availability % · time-to-detection.`

COMPARE

Baseline (default health-check) vs. proposed mitigation — before/after delta.

Initial Research References

Foundational literature anchoring the problem statement, attack model, and proposed defenses.

REFERENCES
§ 5 · 3 CARDS

The following references frame the three intersecting domains this project draws on — container orchestration, denial-of-service attacks, and microservice reliability patterns.

01 CONTAINER ORCHESTRATION

Docker Inc. (2024). Configure healthchecks in Docker Swarm. Docker Documentation.

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. ACM Queue, 14(1).

02 DDOS / AVAILABILITY ATTACKS

Carl, G., Kesidis, G., Brooks, R. R., & Rai, S. (2006). Denial-of-Service Attack-Detection Techniques. IEEE Internet Computing, 10(1).

Peng, T., Leckie, C., & Ramamohanarao, K. (2007). Survey of network-based defense mechanisms countering the DoS and DDoS problems. ACM Computing Surveys, 39(1).

03 MICROSERVICE & CASCADE FAILURES

Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems, 2nd ed. O'Reilly Media.

Richardson, C. (2019). Microservices Patterns. Manning Publications. (Chapter on circuit breakers.)

Aceto, G., Botta, A., Marchetta, P., Persico, V., & Pescapè, A. (2018). Cloud monitoring: A survey. Computer Networks, 57(9).

CEP Topic Approval — questions welcome.

Next milestone — reproduce attack in lab · build detection prototype · measure mitigation delta.